

Agents in Version Control

A plain primer on what an agent runtime has to get right, and how Verlet does it

EMOTION SCIENTIFIC · 21 AUGUST 2026

- | | |
|---|---|
| 1. The thing that breaks | 7. The harness is data |
| 2. Two computers | 8. Where it runs changes nothing about what it may do |
| 3. The record, shown | 9. The field |
| 4. The propagator, or where the name comes from | 10. Status and receipts |
| 5. One checkpoint | 11. Ten questions for any agent system |
| 6. On whose authority? | Appendix: glossary |
-

1. The thing that breaks

Everyone wants agents that grow into the job instead of staying frozen. The market today offers two kinds. Personal agents that you let modify themselves, which regularly brick and need constant babysitting. And hosted agents that are safe precisely because they never change.

The bricking is not because models are bad at self-modification. It is because these agents live in mutable state. The agent edits its own tools, prompts, or memory; nothing records what changed or why; and there is no undo. When it goes wrong, the only fix is to wipe it and start over, which is the same thing as admitting the agent never had a history.

Nobody lets a junior developer, or a bot, touch a production codebase without version control. The same rule has to apply to an agent modifying itself: the agent has to be in version control. Two things go under history, not one. The agent's definition (models, tools, permissions, prompts, procedures: the part that is like source code) and the agent's experience (every run, every model output, every decision: the part that explains why a change was made), with each change pointing at the experience that motivated it.

There is one way this differs from git. Committing cannot be manual, because the committer is the agent itself, and an agent editing its own memory cannot be trusted to

journal the edit. So the runtime has to be built so that acting *is* committing. There is no way to act off the record.

Once that holds, self-modification stops being scary. Every self-change is a commit with a reason attached: diffable, reviewable, revertible. Trying a change is a branch. A bad change is a revert, not a bricked agent.

This primer explains what a runtime has to get right for that sentence to be true, and shows how Verlet, an experimental open-source runtime, does it. The reader it is written for is an engineer or product owner deciding what to build agents on, or what to trust an agent product's claims against. It uses almost no special vocabulary. Where a term is unavoidable, it is introduced as the answer to a problem already on the table.

If any section does not make sense, ask an agent. Each section ends with a prompt. Clone the runtime (`git clone https://github.com/emotionscientific/verlet-kernel`), open Claude Code or any coding agent in that directory, and paste the prompt. The agent answers from the code, not from this document, so what it says is checkable. Boxes marked **Lineage** are optional reading: where an idea came from in older computer science and where this design departs from it. Skip them on a first pass.

To keep everything concrete, one agent runs through the whole document: a customer support agent that reads incoming tickets, talks to customers, and can issue refunds through a payment API. Every piece of it exists in production somewhere today. It moves money, so every question that matters comes up on its own: what happened, why, can we rerun it, and who allowed it.

Discussion

The version-control framing is doing real work, so it is worth saying what it claims and what it does not.

It claims that an agent has two kinds of state that should be one history: its definition (tools, policy, model, sources) and its experience (what it did, what it saw, what it was told). Today these live in different places, under different tools, with no link between them. A config file says what the agent is. A log says what it did. Nothing records that this line in the config exists because of that episode in the log. Putting both on one record is the whole move; everything after this section is mechanism for doing it without lying.

It does not claim that the model is under version control. Model weights live with the provider and change when the provider decides. What the record can pin is which model profile was bound, what was sent, and what came back. That is enough to replay the

system faithfully; it is not enough to reproduce the model's choice, and the next section is about taking that limit seriously instead of papering over it.

Two words from git [25] carry over exactly. A **commit** is an event on the record with a reason attached. A **branch** is a replay from a sequence number with something changed. Two words do not carry over. There is no **merge**, because the record is append-only and the agent is one thread of history, not many; and there is no **rebase**, because rewriting the past is the one thing the design refuses.

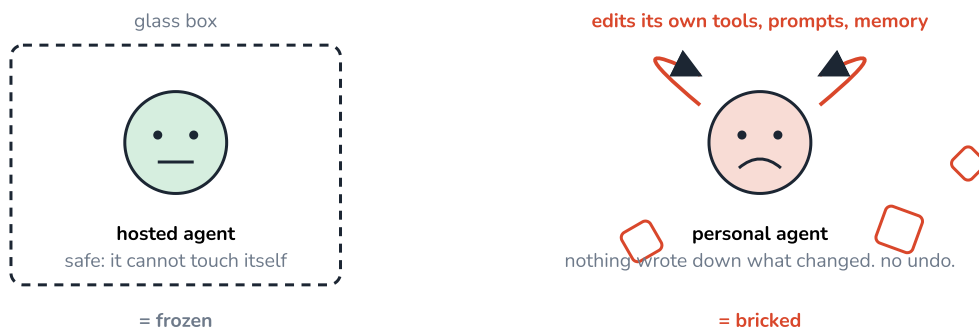


Figure 1.

The fix for both: put the agent in **version control**. Its definition and its whole history, every change pointing at the reason.

ASK YOUR AGENT

I am reading a primer that claims this runtime puts “the agent in version control”: the agent’s definition and its whole history live in one append-only record, and acting is the same thing as committing. In this repository, show me where an agent’s definition is declared, where the record is written, and whether there is any code path that performs an external effect without first appending to the record. Cite file paths.

2. Two computers

“The “I’m sure” computer and the “maybe” computer.”

An agent system contains two kinds of computation that should never be confused.

The first kind is ordinary software. Given the same inputs it produces the same outputs. It can be tested, replayed, and checked. Call it the “I’m sure” computer.

The second kind is the model. Given the same apparent input it may choose a different continuation. Its provider can change it underneath you. Its output depends on sampling, service state, and a model revision you do not control. Call it the “maybe” computer.

Most agent frameworks blur the two. The model’s answer lands in a variable, the variable feeds some code, the code calls a tool, and the whole thing reads as one program. It is not one program. Half of it cannot be rerun.

The right response is not to pretend the model is deterministic, and it is not to give up on guarantees because a model is involved. It is to draw the line between the two computers explicitly, and let the line decide what the runtime must remember and what it may recompute:

- Anything the “I’m sure” computer produced can be thrown away and recomputed from its inputs. Indexes, dashboards, search tables, the context window assembled for a model call.
- Anything the “maybe” computer produced, and anything the outside world did, cannot be reproduced and must be written down the moment it happens. Model outputs. The payment API’s reply. A user’s message. A process exiting.

Every harness already follows half of this rule by instinct: everyone saves what they sent the model, because everyone has learned you cannot recompute a model’s answer. The rule generalizes that instinct to the whole system, and that is where the payoff is. Rerunning yesterday without re-paying for model calls only works if their outputs are part of the record the system runs from. Behavior *between* model calls (routing on a classification, retrieving against an index) is only explainable if the computed state it read is on the record too. A prompt log explains the model. It cannot explain the system, and you cannot restart a system from its prompt log.

There is an older name for this arrangement. In interactive proof systems, Merlin is an all-powerful prover who cannot be trusted, and Arthur is a bounded verifier [1, 3] who can check Merlin’s claims but not produce them. The model is Merlin. The runtime is Arthur. The record is the transcript Arthur keeps, and anything the system can later attest about what the agent did is read from that transcript, never from asking Merlin again.

That also keeps claims honest. A runtime built this way can prove things about ordering, authority, mediation, and whether the record is complete. It cannot prove that what the model said was good or true, and it should not pretend to.

Discussion

The usual objection is that a model at temperature zero is deterministic, so the line is drawn in the wrong place. It is not. Sampling is only one source of drift. Provider-side revisions, batching, hardware, and the serving stack all move the output over time, and you control none of them. “Same input, same output” is a promise the runtime can make about code it ships. It is not a promise anyone can make about a hosted model, so the design does not depend on it.

The second objection is cost: writing down every model output, every external reply, and every user message sounds like a lot of storage. It is less than it sounds. The things that cannot be recomputed are small (text in, text out, an API reply). The things that are large (indexes, embeddings, assembled context windows) are exactly the things the rule says you may throw away and rebuild. The line between the two computers is also the line between what you must keep and what you are allowed to delete.

The deepest consequence is about where judgment is allowed to sit. The “maybe” computer is permitted to propose anything. The “I’m sure” computer is the only one permitted to make something happen, and it does so only against a written proposal it can later show you. Every later section is a variation on this: the model proposes, the runtime disposes, and the disposal leaves a receipt.

LINEAGE optional reading

Interactive proofs, and the thing none of the ancestors had. The prover-and-verifier split comes from complexity theory. Babai’s Arthur and Merlin games (1985) [1] describe a weak, honest verifier accepting claims from an unboundedly clever, untrusted prover, as long as each claim comes with a certificate that is cheap to check. Shamir’s 1992 result, $IP = PSPACE$ [2], says such a verifier, given interaction, can check far more than it could ever compute itself. Read as a design claim: a small deterministic runtime can govern an arbitrarily strong model, provided every consequential step arrives as something checkable. A receipt is that certificate; proof-carrying code [4] makes the same move for untrusted programs. The analogy has a limit worth keeping in view. A theoretical prover finds the accepting path by definition; a model merely samples, and sometimes proves nothing. That is why budgets exist (there is no fixed point to wait for) and why the construction rests on the everyday gap between generating and checking, not on any unproven conjecture. Every older tool this design borrows from, type systems, effect handlers, partial evaluators, assumes its functions are deterministic. Verlet adds one typed slot for a function that is not, the model-backed step, and the rules in sections 3 through 5 are precisely the conditions under which that slot can sit inside otherwise lawful machinery without the whole thing becoming folklore.

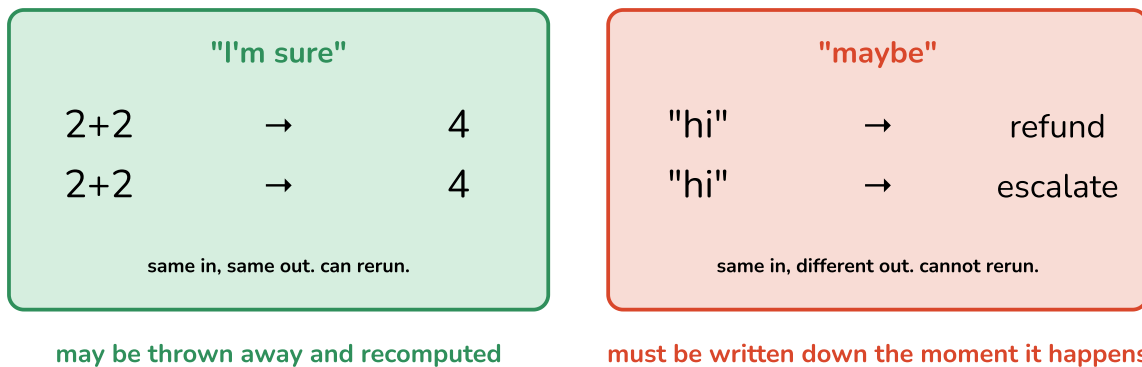


Figure 2.

Indexes and dashboards are green. Model answers, payment replies, and user messages are red. **Red goes in the notebook. Green is a cache.**

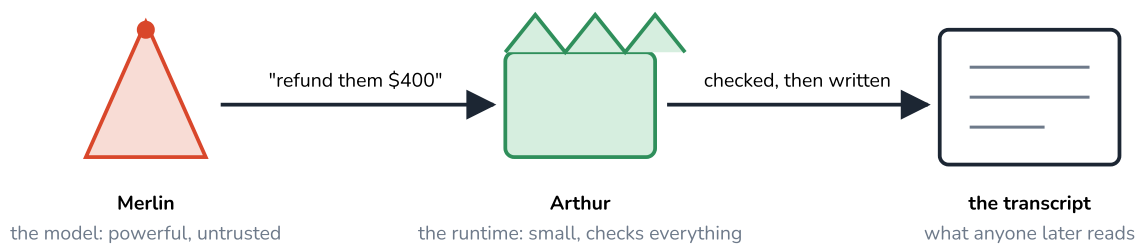


Figure 3.

Nobody ever asks the wizard again what he said. They read the transcript. That is what **attestation** means.

ASK YOUR AGENT

This runtime distinguishes outputs that can be recomputed (indexes, views, assembled context) from outputs that cannot (model completions, external API replies, user messages). Find the type or enum that encodes that distinction, show me how a model completion is persisted as an event, and show me one thing that is treated as a disposable view. Explain the rule the code uses to decide which is which.

3. The record, shown

“Write it down.”

The commitment is the habit everyone was taught and no agent framework kept: everything the system witnesses and everything it concludes goes into one append-only record, and nothing is ever erased. Here is the precise form.

Streams and events

A **stream** is an append-only, ordered sequence of events owned by one scope. The support agent’s conversation with one customer is a stream. So is the control channel that configures the agent, and the audit scope of the tenant that owns it. Streams are never edited. Everything else in the system (every index, every dashboard, every context window) is computed from streams, and nothing else is durable. Order within a stream is the sequence number, assigned on append; there is no clock to disagree with it, which is the old lesson about ordering in distributed systems [24].

There is no session object, no scratchpad, no mutable “agent state” beside the record. An agent does not have state. It has a history, and state is a way of reading it.

An **event** is one immutable fact on a stream, and it has one of two origins.

A **witnessed** event records something the world or the runtime did. A customer message arrived. The payment API returned. A process exited. A model call completed. The runtime saw it happen; that is the event’s authority. Witnessed events are the only way anything enters the system from outside.

A **produced** event records something a component computed. This ticket was classified as a refund request. This conversation was summarized as follows. Every produced event carries **provenance**: which events it was computed from, and by which function at which version. A produced record that cannot name its inputs does not get written.

One rule binds the two: later events may supersede an interpretation; nothing supersedes history. When the classifier revises its judgment, the revision is a new event pointing at the old one. The original judgment, and the fact that the system acted on it for an hour, stay on the record forever.

Receipts

A **receipt** is a recorded explanation of something the runtime resolved. Which human-readable tool name resolved to which immutable operation. Which attachment made a tool visible to this thread. Which policy decision authorized this action. Which sources went

into the context the model saw. Receipts are written under the code and configuration present at the time, and they are never recomputed later under newer code. Recomputing would answer a different question.

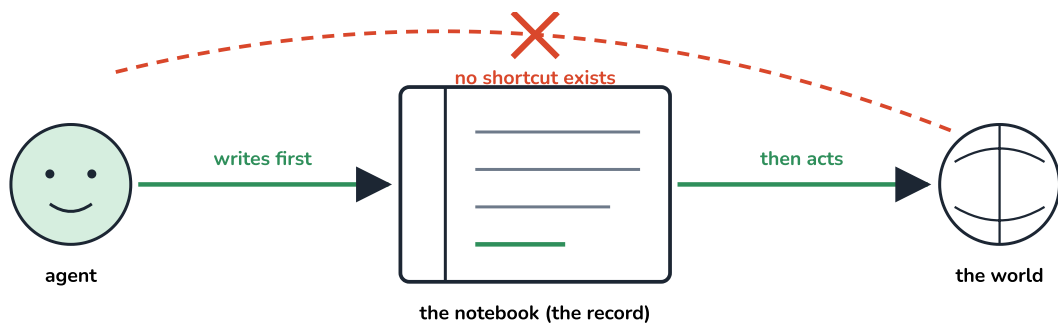


Figure 4.

Not “please log it”. The notebook is the only door. Acting **is** writing it down, so the record cannot have holes.

LINEAGE optional reading

Algebraic effects and handlers. In the effect-systems tradition [12, 13] a function does not perform its side effects; it declares them, and a separate handler, chosen by the caller, decides how each one is carried out. A tool in Verlet is built the same way. The operation declares what it needs (an HTTP origin, a secret by name, a durable write) and cannot reach any of it directly. It asks through an opaque handle, the runtime is the handler, and the handler chooses at call time how the effect is discharged. This is why the same operation runs unchanged on a laptop and on a managed host: same program, different handler stack. Nested calls narrow rather than widen, since a child operation can use at most what its parent had and what the child itself declared. The departure from the textbook is the receipt. Academic effect systems handle effects; this runtime handles them and writes down that it did, with the evidence. An effect with no receipt did not lawfully happen.

What it looks like

Here is the refund, as the record has it. One customer thread, one turn, event kinds as the current runtime names them. Payloads are trimmed.

```
seq  kind                payload (abridged)
41   turn.submitted      principal=operator:support-queue
                                message="Order 8812 arrived broken, refund please"
```


42	context.compile.completed	sources=[instructions@v3, thread:1-41, memory:customer-8812] tokens=2,114
43	session.entry.appended	role=assistant profile=support-default output=tool_call(payments.refund, {order:8812, amount:400.00})
44	tool.call.requested	op=payments.refund@sha256:9f3c... binding=b-201 args_fp=7e11...
45	tool.call.decision	outcome=hold rule=refund-over-250 policy=finance-guard@v7
46	approval.requested	to=principal:finance-oncall facts={op, args_fp, principal, rule}
	... 14 minutes ...	
47	approval.resolved	by=principal:alice outcome=allow
48	tool.call.completed	witnessed: payment_api 200 txn=pay_01J... effect_class=at_most_once
49	session.entry.appended	role=assistant output="Refunded \$400 to your card..."
50	turn.completed	reason=quiescent

Read it top to bottom and every question from section 1 has an answer in the data. What happened: 48. Why: 43, with the context that produced it at 42. Who allowed it: 47, by name, against the rule at 45. What the agent could even see: the sources at 42 and the binding at 44. Nothing here was reconstructed after the fact from three databases and a log file. The runtime wrote each line in order to do the next thing.



Figure 5.

What happened? line 48. Why? line 43. Who allowed it? line 47, by name. Nobody had to reconstruct anything.

Why recovery is free

When the authoritative state is a fold over immutable events, a restart is a read problem. Reopen the streams, rebuild the disposable views, restore the current attachments, continue from the next admissible step.

Take the scene everyone dreads. The process dies between line 48's payment call going out and its result being recorded. On restart the runtime finds a `tool.call.requested` at 44 with an allow at 47 and no completion. The operation's declared effect class is at-most-once. So the runtime does not retry. It queries the payment API for the transaction by the request's fingerprint, records what it finds as a witnessed event, and either continues or fails loudly with an explicit interruption. It does not guess, and it does not refund twice.

(In a real production system you should also make the refund API itself idempotent, keyed on the request fingerprint. Then the operation can be declared idempotent, and recovery becomes a safe retry instead of a lookup. The runtime's job is to know which of the two it is dealing with, and to record which one it did.)

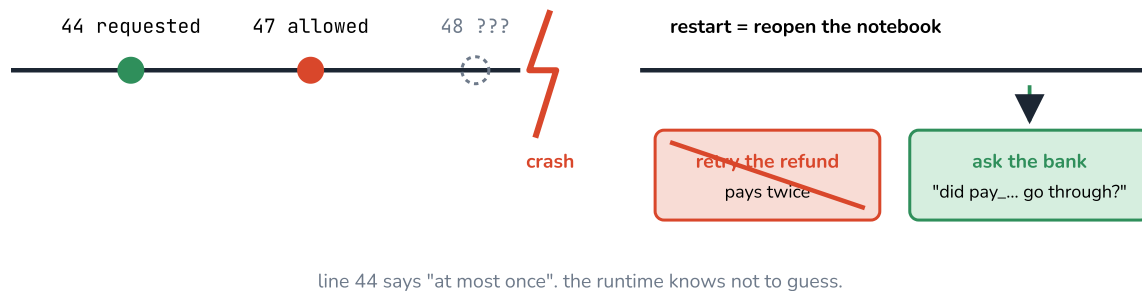


Figure 6.

Recovery is **reading**, not re-thinking. No model call, no double refund. (And yes: make the refund API idempotent too.)

Compare that with a harness whose state is in memory and whose log is a side channel. After the crash it has a transcript and a hope. The honest options are to replay the whole conversation through the model again (paying again, and possibly getting a different decision) or to ask a human what happened.

Why replay is exact

Because every model output and every external reply is on the record, the deterministic parts of yesterday can be rerun against it exactly, with no model calls. You can branch at

line 45, swap in `finance-guard@v8`, and see whether it would have held the refund, for the cost of reading the stream. This is what makes a change to the agent testable before it ships, which is the whole point of putting the agent in version control.

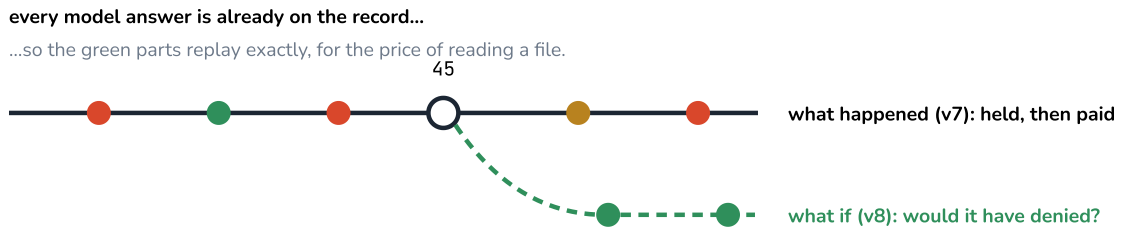


Figure 7.

This is how a change to the agent gets **tested before it ships**. Version control needs branches; now the agent has them.

Why observation is not enough

A monitoring product watches an agent from the side and infers what it did. That is useful, and it is not the same thing. Important steps may be missing, sampled, or disconnected from the state that governed them. The reconstruction is a best guess about the system.

A runtime where acting is committing generates the evidence while doing the work, because it could not have done the work otherwise. Monitoring products stay useful as views over that record. They just start from a stronger source: the facts execution itself required. Section 6 returns to this line, because it is the one that matters most when someone has to certify an agent.

Discussion

Three things people ask at this point.

Can the record be wrong? It can be incomplete if a process dies between a side effect and the witness for it; that is the crash scene above, and the answer is idempotent effects plus replay, not a promise that it never happens. What the record cannot be is silently edited. Events are appended under the authority of whoever produced them, and the runtime's own attestations are marked as such. A wrong entry is corrected by a later entry that says so. Nothing is overwritten.

Is this just logging with extra steps? The pattern is event sourcing [23], and the difference is which way the dependency points. A log is written after the fact, from state that lives elsewhere; if the log is lost the system keeps running. Here the record is the state. A thread's tool list, its budget position, its held actions, all of it is computed by reading the record forward. If the record were lost there would be nothing to run. That is what makes replay exact and recovery free, and it is the property that a log, however thorough, cannot offer.

What is a receipt not? It is not a proof that the decision was right. A receipt for a policy decision says which policy, which inputs, which outcome, under which code. Whether the policy was a good one is a question for the people who wrote it, and the receipt is what lets them ask it with the facts in hand.

ASK YOUR AGENT

Walk me through one turn of a thread in this runtime as a sequence of events: the kinds `turn.submitted`, `context.compile.completed`, `tool.call.requested`, `tool.call.decision`, `approval.requested`, `session.entry.appended`, `approval.resolved`, `tool.call.completed`, `turn.completed`. For each, where is it appended, what is in the payload, and which ones carry provenance pointing at earlier events? Then show me what resume does after a crash: where does it reopen the stream, and how does it decide whether an in-flight tool call is retried, looked up, or failed? Find the effect class concept (at-most-once vs idempotent) in the code.

4. The propagator, or where the name comes from

“An agent is a loop wired into a record.”

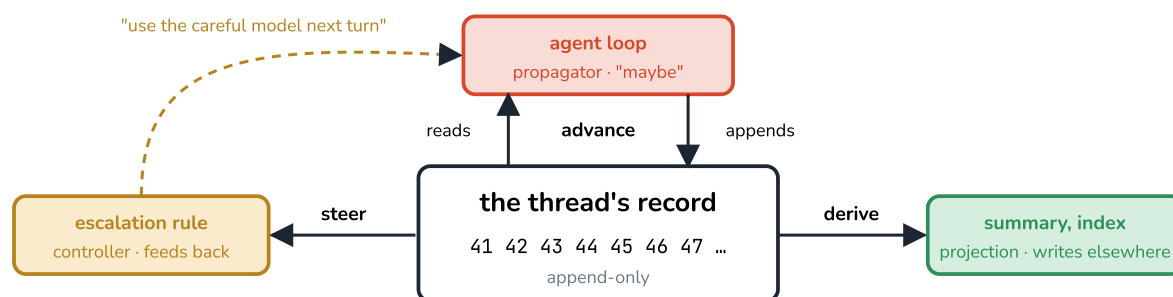
Section 3 showed the record. This section says what runs around it, and the answer is smaller than most frameworks make it.

Everything that acts on a thread is a **coupling**: a function with three declared legs. What it reads (a selector over streams). Where its output goes. What activates it. A coupling has no authority beyond those three declarations; it reads what its selector names, writes where its sink names, and fires when its trigger fires. Every piece of machinery in the runtime, the agent loop included, is one of these.

Couplings come in exactly three kinds, and the kind is decided by one question: where does the output go?

- If the output goes back into the stream it read, the coupling **advances** the system. It is a **propagator**. The agent loop is the privileged one: it reads the thread's record, calls the model, and appends what came back to the same record, triggered by a turn being submitted. Thread, model, thread.
- If the output goes somewhere else as a derived result, the coupling **derives**. It is a **projection**: a summary, an embedding, a classification, an index feed. Projections are allowed to lose information; that is their job. What they may not do is hide what they read, so every projection's output names its inputs.
- If the output goes into a control stream, something that changes how future turns run, the coupling **steers**. It is a **controller**: switching to a more careful model when a conversation escalates, gating a tool, deciding when to compact context, routing an inbound message to a thread. Products call these hooks. They are feedback.

Advance, derive, steer. The three are exhaustive because the question is: output either returns to its source, or lands elsewhere as interpretation, or feeds forward into control. Try it on your own stack. The support agent decomposes with nothing left over: the loop is the propagator; the ticket classifier, the summarizer, and the embedding feed are projections; the escalation rule and the compaction trigger are controllers.



every one of these is a coupling: reads declared, writes declared, trigger declared.

only the loop writes back into its own source. that is what makes it the propagator.
integrator · analysis · thermostat, in molecular dynamics words.

Figure 8.

One record, three kinds of function around it. The loop **advances** it. Projections **derive** from it. Controllers steer what the loop does next. The agent is the loop and nothing more.

Why this is the whole runtime

The payoff of having only one kind of thing is that nothing about the agent is special. An agent is a propagator wired into streams. A sub-agent is another propagator on another thread. A workflow is a propagator whose next step is chosen by a script instead of a

model. A “memory system” is a projection that writes and a context source that reads. The runtime does not ship an agent abstraction, a workflow abstraction, and a memory abstraction that happen to interoperate; it ships streams and couplings, and those words are names for wiring patterns.

It also says where the two computers from section 2 live. A coupling is either deterministic or **chaotic**, meaning its output cannot be reproduced from its inputs. Model-backed couplings are chaotic by definition. The rule from section 2 becomes a rule about couplings: a chaotic coupling’s output must land on a stream as an event, because there is nothing deterministic to regenerate it from; a deterministic coupling’s output may be a view instead, cacheable and deletable. The agent loop is chaotic, which is why its every output is on the record and why the checkpoint in the next section sits directly on its output.

One more property falls out. Nothing in the system can start itself. Couplings fire on events; derived events trace, through provenance, to the events that produced them; and that chain only ends at a witnessed event, something that came from outside: a user’s message, a timer the operator set, an API reply. “The agent decided on its own to start doing something” is not a scenario the runtime can represent. Every activation chain begins with someone.

Discussion

The word propagator is borrowed from molecular dynamics, and the borrowing is the origin of the project’s name. In a molecular dynamics simulation, the **integrator** advances every particle one step from the current state; the Verlet integrator (Loup Verlet, 1967 [17]; the velocity form in common use is from 1982 [18]) is the standard one, and it has a property that matters here: it is time-reversible and it conserves what it should conserve over long runs, because it is built around the trajectory rather than around a single force evaluation. **Analysis functions** are computed over the trajectory without touching it. **Thermostats** and biases read the trajectory’s history and feed back into the dynamics to steer them. Advance, derive, steer, one-to-one [20]. (The word is the integrator sense, not the propagator networks of Sussman and Radul [26], which are a different idea.)

The creator’s training is in computational chemistry and physics, and Verlet was conceived by noticing that the abstractions that make a molecular simulation trustworthy (one trajectory as the truth, an integrator that only ever advances it, analysis that never writes into it, feedback that is itself a function of the history) are the same abstractions an agent runtime needs and usually lacks. The mapping is a mnemonic, not a proof; the definitions above stand on their own. But the name is a reminder of the discipline: the thing that advances the system is one small, well-understood function, and everything else either reads or steers.

Metadynamics, or why agent memory is a thermostat. Molecular simulations have a standard problem: the system gets stuck in a valley of its energy landscape and never visits the rest. Metadynamics (Laio and Parrinello, 2002 [19]) fixes this with a bias that is a function of the trajectory's own history. Every so often it drops a small hill where the system currently is, so that places already visited become less attractive and the system is pushed toward what it has not yet seen. Two things about that construction carry over. First, the bias reads the history and writes into the dynamics; it is a controller in the sense above, and agent memory has exactly that shape: a projection over past turns whose output changes what the loop does next. Second, and this is the part most memory features miss, the bias is a deterministic function of the trajectory. Given the record, you can recompute what the memory was at any step, which is what makes its influence explainable after the fact. The lexicon's line for this is short: agent memory is metadynamics, history-dependent bias shaping future evolution. The departure is the usual one. A thermostat does not need receipts, because the simulation has no adversary and no auditor. Here the feedback leaves events with provenance, so "why did it pick the careful model on turn 12" is a query rather than a guess.

ASK YOUR AGENT

This runtime classifies every function that acts on a thread as a coupling with declared reads, writes, and trigger, and sorts couplings into propagator, projection, or controller by where the output goes. Find the coupling definition and the three roles in the code. Show me the agent loop as the privileged propagator (thread in, model, thread out), one projection, and one controller, and show me where a chaotic coupling is forced to write events rather than a view.

5. One checkpoint

"What may happen next?"

The industry sells workflows and agents as different products. A workflow follows a graph someone drew. An agent lets the model decide. Teams end up running both, on two stacks, with two logs, and a hand-off between them that nobody can audit.

Operationally they were never different things. Every step the system takes, whether a model call, a tool invocation, a spawned child, or the end of a turn, is a continuation of

some thread. Put one checkpoint in front of all of them. At each step it asks one question: given this thread's history, which continuations are admissible next? It picks one, and it writes the decision to the record as an event like any other.

Two properties make that checkpoint more than a dispatcher. First, there is no side door: every continuation the runtime executes passed the checkpoint, because the checkpoint is where execution is obtained. Second, the decision itself is history. The record shows not only that the refund tool ran, but that at step 45, under `finance-guard@v7`, the admissible set was `{hold, deny}` and `hold` was selected. Post-mortems stop reconstructing what the system was choosing between.

The checkpoint's question is answered by a **policy**, a function from the thread's history to the set of admissible next steps. Policies come in two modes, and the pair is the whole taxonomy.

In **strict** mode the policy is deterministic. Only the moves it names are admissible, and replaying the record re-derives every decision it ever made, exactly. A strict policy is what the industry calls a workflow.

In **adaptive** mode the model selects the continuation, inside a declared envelope of tools and budgets, landing on the same record through the same checkpoint. An adaptive policy is what the industry calls an agent. The everyday agent loop ("the model decides what to do next, with its tools available") is the simplest adaptive policy: admit the whole envelope at every step. It is the correct default and the weakest interesting setting.

Made per step, the choice stops being an architecture and becomes a dial. The support agent's refund path runs strict: verify the order, check the amount, hold above a threshold. The conversation around it runs adaptive. Same thread, same record, same runtime.

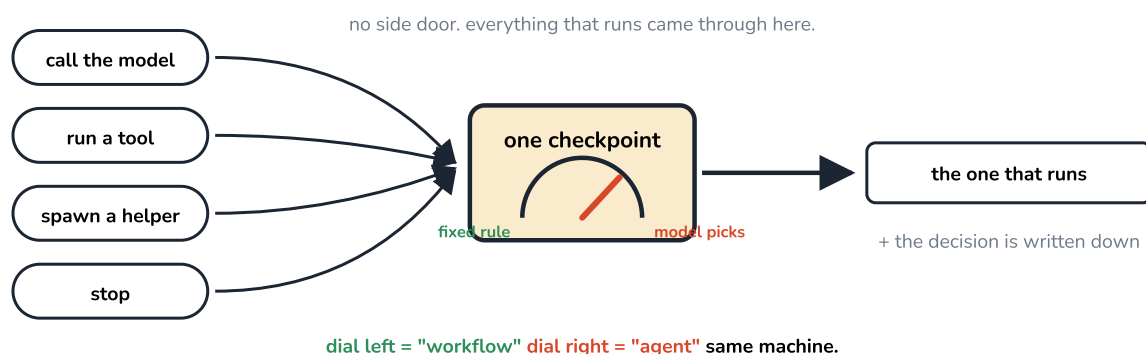


Figure 9.

Workflows and agents were never two products. It is one dial, turned **per step**: refund path strict, chit-chat loose, same notebook.

Budgets instead of assumed convergence

A deterministic workflow can often prove from its graph that it terminates. A model-backed loop cannot. So the runtime enforces explicit limits on turns, depth, time, cost, and triggered work, and a turn ends when the policy reaches quiescence or a budget stops it. Budgets are runtime mechanisms, not instructions in a prompt, and they leave evidence: the record says whether work completed, was stopped by policy, ran out of budget, or was canceled by a named person.

Discussion

“Strict” and “adaptive” are often presented as two products: a workflow engine and an agent framework. Treating them as one dial, set per step, changes how a team builds. Nobody has to decide up front whether a task is “an agent problem” or “a workflow problem.” A support flow can be strict for the refund step (three checks, in order, no improvisation) and adaptive for the conversation around it. The mix is a property of the policy bound to the thread, and the record shows which setting governed each step.

Budgets deserve their own sentence because they are easy to dismiss as configuration. They are the only honest answer to a loop with no fixed point. A deterministic workflow stops because its graph runs out. A model can always propose one more step, and “it will probably converge” is not a guarantee anyone should ship. A budget turns an open question into a bounded one, and the record saying *why* a turn ended (done, held, out of budget, canceled by whom) is what an operator reads first when something went wrong.

Improvisation becomes procedure

Because a policy is data, an agent can write one. After the support agent has improvised the same three-step check on fifty broken-order tickets, the sequence can be published as a strict policy with provenance pointing at the fifty episodes, and bound for future runs. The model stays available for the novel case; the repeated case gets cheaper, faster, and inspectable. This is the path from improvisation to procedure, and the record supplies the reasons behind the diff.

Partial evaluation and the Futamura projections. Partial evaluation asks of every part of a program: when does this become known? Anything known early can be specialized away, leaving a smaller residual program for what is still open. Futamura's 1971 observation [5] was that applying this to an interpreter and a program yields a compiled program, and applying it again yields a compiler. Improvisation becoming procedure is the same move one story up. The running agent is an interpreter; what it interprets is the task, in natural language. Repeated interpretive work (the same three checks on fifty tickets) is the part that has become known, and condensing it into a strict policy is specialization with a receipt. The deterministic machinery below (manifest in, bound harness out) is the first projection in the textbook sense; the specialization above it is chaotic, because a model proposed it, which is exactly why it lands as a proposal on the record and passes the checkpoint rather than being assumed. There is even a gauge. Jones optimality [6, 7] asks whether specialization removed the entire interpretive layer; here the share of a thread's steps that still went to the model, per recurring task, is that overhead, readable straight off the record. Inference systems already do this by hand: prefix caching is the model specialized on a fixed context, and fine-tuning is weights specialized on history. The rule this ancestry leaves behind is that specialization must be auditable. A residual program with no receipt for how it was produced is folklore.

ASK YOUR AGENT

The primer says every continuation (model call, tool call, child thread, end of turn) passes one admission checkpoint, and a policy answers "what may happen next" in either a deterministic mode or a model-selected mode. Find that checkpoint. Is there any way for a continuation to execute without passing through it? Show me where budgets (turns, depth, time, cost) are enforced and what event is written when a budget stops a turn.

6. On whose authority?

"A tool that is not attached does not exist."

Here is the failure that governance products are sold to prevent. The model proposes a refund of \$40,000. Nothing checked whether it could. The money moves. Afterward, the team discovers the refund tool was reachable from a prompt nobody had reviewed, through a path nobody had drawn.

The dangerous property was never that the model proposed it. Models propose things constantly. The dangerous property is an unmediated route from the proposal to the world. Two laws close that route, and both are compiled into the runtime rather than configured on top of it.

Attachment. An operation that has not been attached to a thread has no tool surface on that thread. Not “is denied”; does not exist, is not in the model’s tool list, cannot be named. Attaching is an event on the record (section 7 covers it). So the question “what could this agent have done?” is answered by folding the thread’s attachments, not by reading prompts and hoping every branch was found.

Mediation. Every external effect passes through one runtime-mediated surface. That surface is where a configured policy can allow, deny, or hold the action, and where the decision is recorded and tied to the action it governed. A denial is visible to the agent so it can choose another path. An approval authorizes exactly one fingerprinted action, not a class of them.

Behind both sits a **principal**: the human or service identity on whose authority the work entered. Line 41 of the excerpt names one. The agent is not itself a principal. It acts within authority that arrived from outside, and every effect traces back to the name that asked for it.

Why rules alone cannot do this

There is a reason rules-first governance keeps failing at agents, and it is older than agents. Ashby’s law of requisite variety [8, 9] says a regulator can only control a system whose variety it can match. A policy written in advance cannot match the variety of what a model will propose; every team that has tried to enumerate allowed actions has watched the list go stale in a week.

The regulator that can match model variety is not a longer list. It is the pair above: a complete record, and one choke point that every effect must pass, where a decision of any sophistication can be made, by a rule, a person, or another model, at the moment the specific action is known. The runtime supplies the choke point, the hold, the resume, and the record. The judgment is pluggable.

Discussion

Attachment as the unit of authority takes some getting used to, because most systems make authority a property of the agent (“this agent is allowed to refund”). Here it is a property of the thread’s history: at sequence 12 a refund tool was attached, with this configuration, on this person’s authority, and it has not been detached since. The difference

shows up the first time someone asks “since when could it do that, and who said so.” A role-based answer requires reconstructing who changed the role and when. An attachment-based answer is one query.

Mediation is the other half. It is tempting to think of the checkpoint as a firewall that inspects traffic. It is closer to a notary. The action does not happen and then get checked; it is proposed, held, decided, and only then performed, and the decision is written before the effect. This ordering is what makes “every consequential action was decided” a claim about construction rather than a claim about coverage.

LINEAGE optional reading

Cybernetics. The regulation vocabulary here (controllers, feedback, budget-enforced quiescence) descends from cybernetics [10], and Ashby’s law of requisite variety is the one theorem doing load-bearing work: a regulator must have at least as much variety as the thing it regulates. A fixed checklist dies when the model’s behavior outgrows it; a checkpoint that sees the specific proposed action, with the whole record available, grows with the model it governs. The interesting part is what cybernetics lacks. It has no notion of truth (an authoritative, append-only account of what happened), none of attestation (who witnessed a fact versus who produced it), and none of permission (what a part *may* do, as opposed to what it *can* do).

Every Verlet deployment is a cybernetic system; almost no cybernetic system is a lawful Verlet one, because those three commitments are what is left after the dynamics are accounted for. Even second-order cybernetics [11], which put the person doing the observing inside the system, left observation as a signal. Here the act of observing is itself a durable event with provenance. The regulator must leave receipts: Arthur, not merely a thermostat.

Governance should keep a path to yes

A control that only blocks moves pressure into bypasses. Useful governance offers resolution: say why the action was denied and which rule applied; let a person approve with a bounded fact set; let the agent revise the action; detach a failing capability; record an authorized exception; resume held work after a decision. Line 46 through 47 of the excerpt is that path in miniature: fourteen minutes of a human deciding, with the thread parked and nothing lost.

Bounded disclosure

The person approving a refund rarely needs the whole transcript. A decision request carries a typed fact sheet: the action, its argument fingerprint, the principal, the binding, the

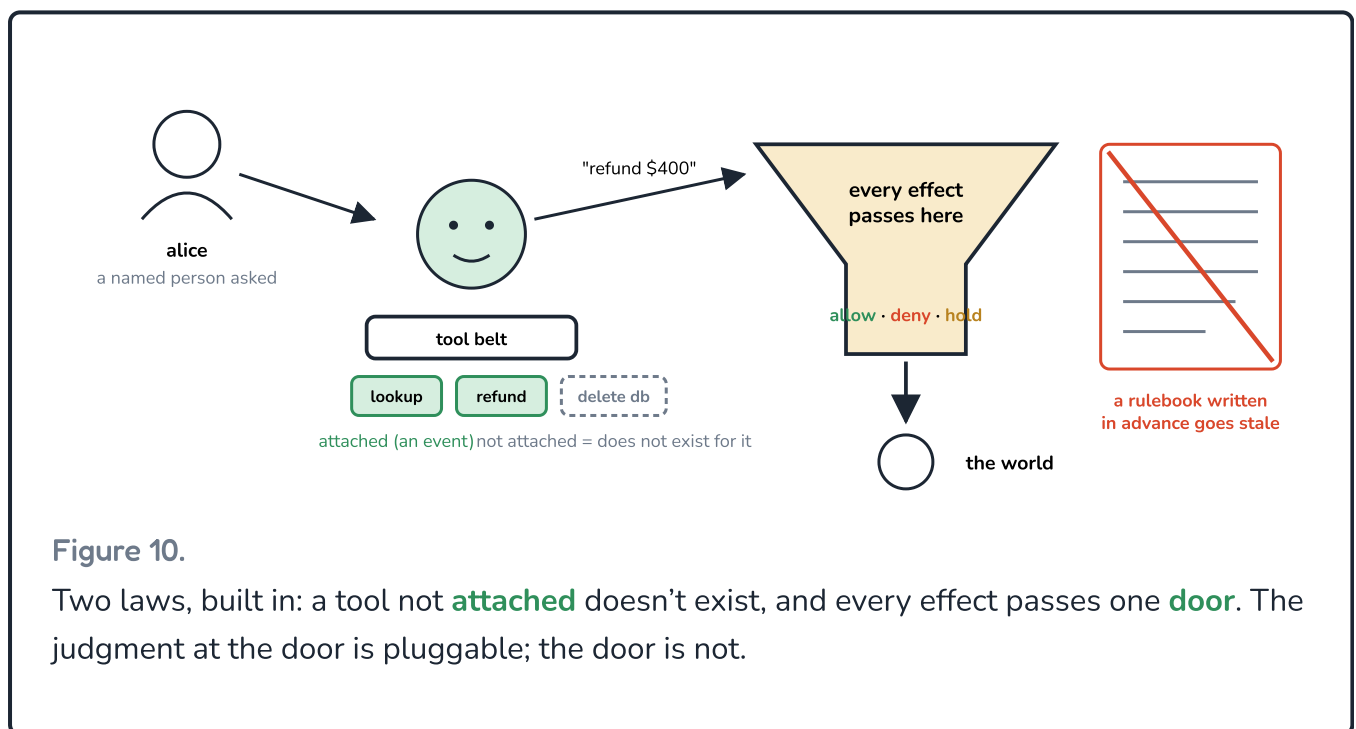
rule, and a bounded slice of history. Any explanation the agent wrote is marked as untrusted content. Less private context leaves the thread, and model-generated text cannot quietly present itself as authority.

By construction and by observation

This is where the two-computers cut pays off for anyone who has to certify an agent. There are two ways to know what an agent did.

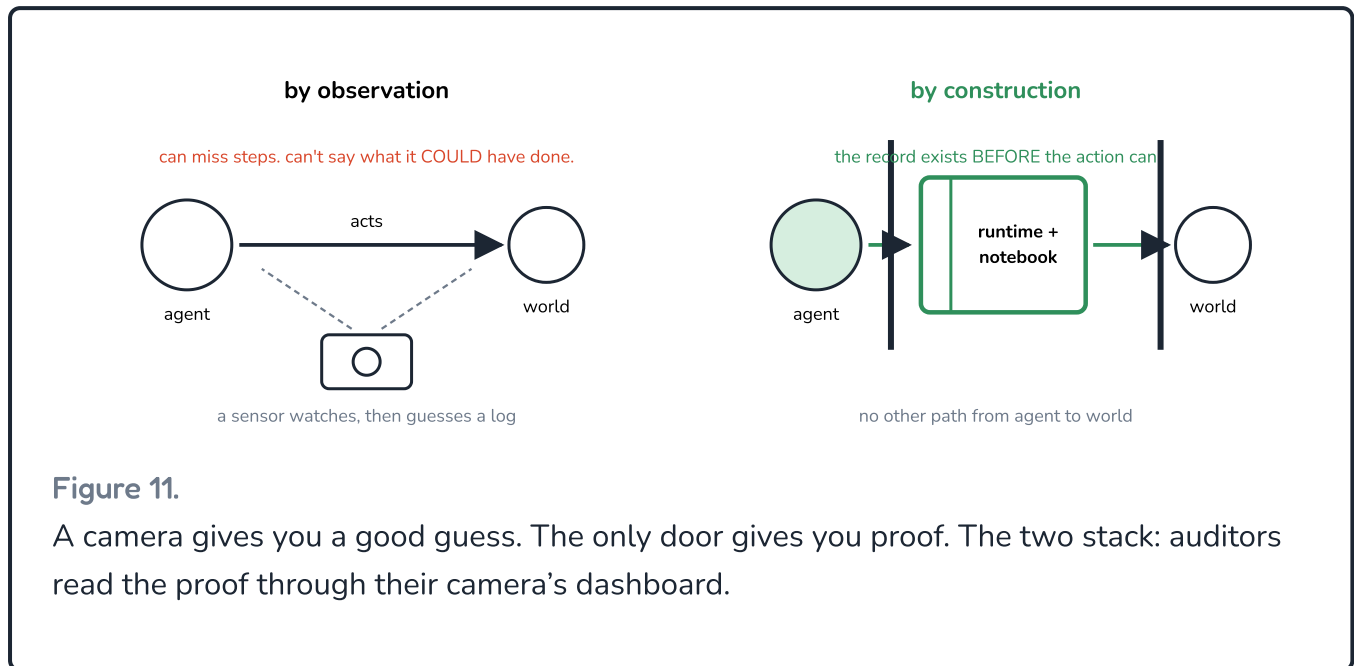
By observation. A sensor sits beside or in front of the agent, captures what it can see, and reconstructs a chain of custody. This is what most governance products do today. It works on any agent, which is its strength, and it is an inference, which is its limit. It cannot know what the agent could have done, only what it saw it do.

By construction. The runtime is the only way to act, so the record is not a reconstruction of the work; it is the work's precondition. The evidence exists because the action could not have happened without it. Attachment answers "what could it do," mediation answers "who allowed it," and the record answers "what did it do," all from the same source.



The two compose. An observation product consumes a by-construction record as its strongest input, and maps it to the controls and frameworks its customers need. The integration is two seams: policy decisions in (the runtime sends a structured request and records the answer), and runtime evidence out (events and receipts exported with stable identities and links back to the source record). The runtime stays the place where a decision becomes an enforced and recorded action.

Honest status: attachment and mediation are compiled into the shipped runtime. The pluggable policy router, with hold and resume behind an external service, is design-settled and not yet shipped. Section 10 keeps the full ledger.



ASK YOUR AGENT

Two claims to check. First: an operation that is not attached to a thread has no tool surface on that thread, meaning it does not appear in the model's tool list at all. Show me where the tool list for a model call is computed from attachments. Second: every external effect passes through one runtime-mediated surface where a policy can allow, deny, or hold it. Show me that surface, and show me how a hold is parked and later resumed. Also show me where the principal (the identity on whose authority work entered) is recorded on a turn. Tell me honestly which parts of the pluggable policy router are implemented and which are not.

7. The harness is data

"Don't put it down, put it back."

Section 1 promised the agent in version control. Sections 3 through 6 built the record, the loop, the checkpoint, and the authority model. This section is where the promise is kept.

Everything that makes up the agent's harness (which tools it has, which policy governs it, which model profile it runs, which sources feed its context) is declared data, and every

change to that data is an event on the thread's record.

A tool becomes available through an **attach** event and unavailable through a **detach** event (`binding.attached` and `binding.detached` in the record). The set of tools the agent has at any moment is not stored anywhere; it is the fold of those events up to now.

Configuration for a tool (which secret it may use, which network it may reach) rides on the attach event, so “what could it reach, and since when” is one query over the record.

Changes are prospective and take effect at a turn boundary. A turn already in progress keeps the snapshot it started under. A registry entry changing tomorrow does not alter the contract a running turn started with today, because execution resolved the human name to an immutable, content-addressed operation and recorded the resolution. Rollback is a new recorded change, not a rewrite of the past.

Self-extension through the same gate

Now the 3am scene. The support agent, working a ticket, discovers it needs a lookup it does not have. It writes the operation (a small script, a published immutable package), and requests attachment.

That request is an action like any other. It reaches the checkpoint. A policy decides whether the change widens the agent's authority, merely adds structure inside authority it already has, or needs a human. The attach event records who requested the change, which decision authorized it, and which immutable operation became available. At the next turn boundary the agent's tool list reflects it.

Nothing new was needed for this. The same record, the same checkpoint, the same mediation, the same receipts that govern a refund govern the agent changing itself. That is the entire mechanism, and it is why the bricking story from section 1 stops: the change is a commit with a reason, the episodes that motivated it are on the record next to it, and detaching it is one more event.

Version control, finished

Put the pieces together and the git analogy closes. The agent's definition is data under history. The agent's experience is the same history. Each change points at the experience that motivated it. Trying a change is a branch at a sequence number with a different policy bound. A bad change is a detach. Nothing is ever off the record, because the record is how anything happens.

Discussion

“The harness is data” can sound like “the harness is a config file,” which every framework already has. The difference is in three properties the config file lacks.

First, the harness is never edited in place. It is the fold of attach and detach events, so every version of it that ever existed is still readable, and each change is next to the reason for it.

Second, the harness is resolved, not declared. A manifest proposes tools by human-readable name; what the thread actually runs against is the set of immutable, content-addressed operations those names resolved to at bind time, with the resolution receipted. Exporting the harness means reading that resolved set, which is why two machines can compare harnesses and find the exact line that differs.

Third, the agent can change it through the same gate that governs everything else. This is the property that makes self-extension safe enough to allow at all. There is no privileged “reconfigure” path that bypasses the checkpoint, and so there is no way for the agent to quietly become something other than what the record says it is.

LINEAGE optional reading

The program is data; declarations select, they do not compute. Two older schools meet here. The first is the initial-encoding school in functional programming [14, 15]: represent the computation as a value, keep the interpreter swappable, and enforce the laws in the constructors that build the value. The manifest is that value and the kernel is its interpreter. The constructors are the compile and bind steps: names resolve to immutable hashes, unknown references are rejected before anything runs, every resolution gets a receipt. Hot-swap works because reconfiguration hands the interpreter a new term; it never patches the interpreter. The rule this leaves behind is that if a behavior cannot be read off the declaration plus the receipts, it is hidden interpreter state, and a bug.

The second school is template metaprogramming. Declarations are templates and bind is instantiation, done fully before any run; one operation definition instantiates as a CLI command, an HTTP route, a model-visible tool, and an MCP export, each faithful to the same definition; and the exported harness is the expansion, the post-instantiation artifact that shows what the declarations became. C++ templates became an accidental programming language with no laws, no budget, and unreadable expansion errors. Verlet caps the declaration layer on purpose. Declarations select and arrange among registered pieces; anything that computes is an operation, where attachments, provenance, and budgets apply.

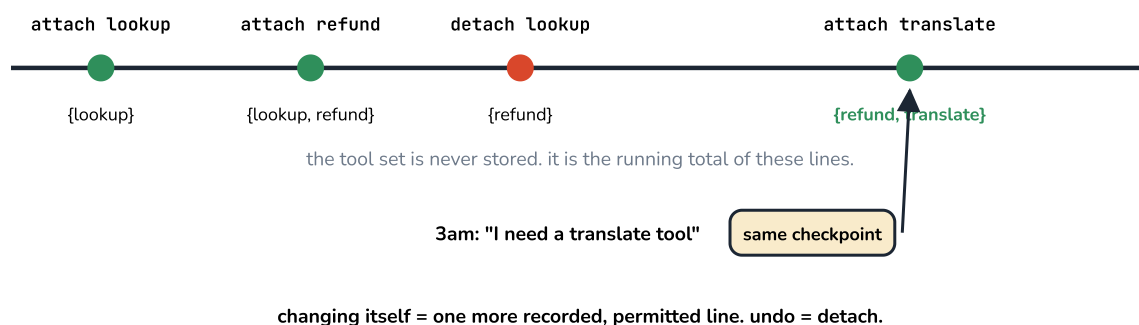


Figure 12.

The agent's harness is **data on the record**. Self-modification goes through the same door as a refund. That is the whole trick.

ASK YOUR AGENT

Show me the attach and detach events for operations on a thread, and the code that folds them into the current tool set. Confirm the tool set is derived from the record rather than stored as mutable state. Then show me how a change takes effect at a turn boundary rather than mid-turn, and how execution resolves a human-readable operation name to a content-addressed immutable identity and records that resolution. Finally: can an agent publish a new operation and request that it be attached to its own thread? Trace that path and say which policy decides.

8. Where it runs changes nothing about what it may do

“Mind your own business.”

Authority lives in attachments on the record. Placement is a separate decision: which machine, which sandbox, which worker. Moving a thread from a laptop to a managed host, or from one worker to another after failure, does not change what it may do, because nothing about authority was ever stored in the machine.

Two consequences. First, many customers' agents can share one runtime host, each inside its own fence, with the fence defined by attachments and principals rather than by process boundaries. Second, a sandbox and a policy decision answer different questions, and the

record says which guarantee applied. A strong sandbox does not decide whether an action was authorized. A strong policy decision does not confine arbitrary code. Agent systems need both.

Discussion

The split between placement and authority is what makes the rest of the document portable. If “what it may do” were a property of the machine (this container has the payment key, so whatever runs in it can refund), then moving the agent would mean re-deriving its authority, and a multi-tenant host would be one misconfiguration away from a cross-customer refund. Keeping authority in attachments means the machine is dumb on purpose. It runs what it is given, against the record it is handed, and the question “could this thread reach that system” is answered by the record regardless of which worker happened to be executing.

The sandbox point is the one most often collapsed. A sandbox answers “what can this process physically touch.” A policy decision answers “was this action authorized, by whom, under which rule.” A system with only the first cannot explain its own behavior; a system with only the second cannot contain code it did not write. Both guarantees are worth having, and the record should say which one applied to each effect, because they fail in different ways.

The developer loop

For the person building the agent, the whole thing reduces to one loop that uses the same contracts locally and managed.

Define. A folder: instructions, tools, resources, model profiles, the context policy, what secrets and networks are needed. Readable as a description of the runtime object, checked into git, handed to another runtime without a tutorial.

Plan. Resolve the proposed shape without running it. Which immutable operations, which secrets, which network origins, which resources, which policy decisions will be required, and which capabilities the chosen placement cannot honor. A system that cannot explain its requested powers before execution forces the reviewer to audit code and hope.

Run. Locally, through a CLI, terminal console, RPC client, MCP or ACP surface, all presenting the same operations and the same thread history. The local run produces the same categories of events and receipts a managed run will.

Publish. Operations by content identity. A mutable name may point at one, but execution resolves the name and records the resolution. Publishing a tool does not make it visible to any agent; a later attach does, within a scope, under policy.

Promote. Pick a placement and the organization's bindings. The managed host supplies identity, tenancy, secrets, quotas, and operations. The definition stays inspectable and exportable, which is what avoiding lock-in means in practice.

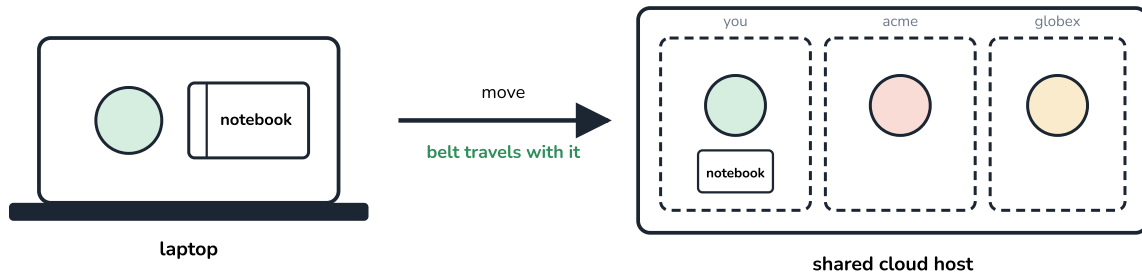


Figure 13.

What an agent **may** do lives in its notebook, never in the machine. So it can move, and many tenants can share one host, each inside its own fence.

Operate. Inspect current attachments, the hashes that produced them, outcomes, decisions, outstanding work, failures, placement, and the provenance of any change. Revise prospectively. Revoke by detaching.

ASK YOUR AGENT

The primer says placement (which machine or sandbox runs a thread) is a separate decision from authority (what the thread may do), and that a thread can move without changing what it may do. Show me the placement decision event and where it is recorded. Show me what state travels with a thread when it is placed remotely and what does not. Then show me how multiple tenants or instances share one host and what keeps them apart. Also list the interfaces (CLI, daemon, RPC, MCP, ACP) and confirm they expose the same operations and thread history.

9. The field

Agent systems touch several product categories that claim the same nouns while owning different things. Four responsibilities keep them apart.

Responsibility	Owns	Examples
Authoring and distribution	definitions, packages, catalogs, registries	what should exist
Governance and control	rollout intent, organizational policy, approvals	what should be allowed
Runtime execution	threads, attachments, mediation, recovery, placement, receipts	what ran
Evidence and assurance	search, investigation, control mapping, reporting	what it means to an auditor

The products may be bundled. The contracts stay distinct, and the most important line is between desired state and executed state. A catalog says version 3 is approved for the finance team. A policy service says this action is allowed. The runtime record says whether version 3 actually started, which attachments it received, whether the decision reached the mediated action, and what followed. Both matter; they should be joinable and never collapsed. When they differ, the difference is the incident.

LINEAGE optional reading

Traits, typeclasses, and components. The kernel defines very few contracts: how an operation is invoked, how context is assembled, and the shape of a coupling. Product words such as “memory,” “skill,” “hook,” and “subagent” are never implemented against directly. Each is a name for a pattern over those contracts. Memory, for example, is anything with a writer that produces events and a reader that feeds context; nothing declares itself to be a memory, and anything with that shape is one. This is the typeclass move [16] (one contract, many derived names, instances by structure rather than by inheritance) and the entity-component move from game engines, where “supports memory” is a query over components, not a subclass. The rule it leaves behind is what makes the four-responsibility table above stable: a new product word must lower to existing contracts, and a product word that needs a new kernel primitive is a design failure until proven otherwise.

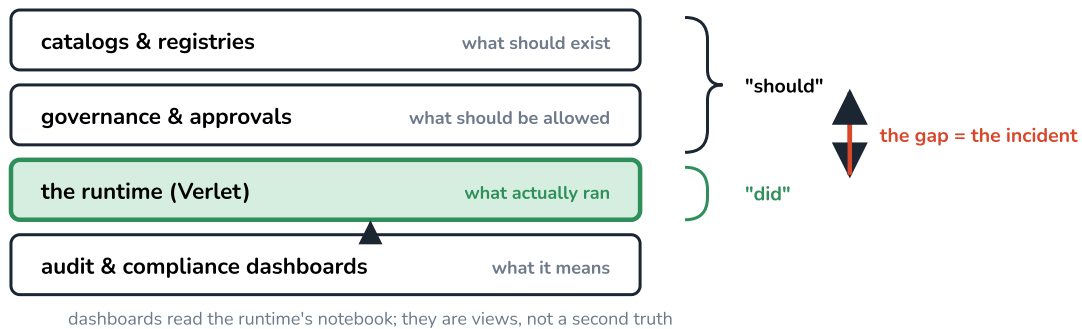


Figure 14.

Keep “should” and “did” joinable but separate. When they disagree, you have found the incident, as data, not as a week of forensics.

Cordis and DeepSeek Harness

Two recent systems show the field moving the same direction from a different starting point, and they deserve a fair page.

[Cordis](#) [30, 31] is a TypeScript meta-framework for programs whose components arrive, leave, or change while the process is live. A component that changes shared context supplies an inverse, and the runtime tracks what it would take to withdraw it. Components declare what they require and provide, and Cordis reacts when providers appear or disappear. Its [paper](#) develops this into a calculus of components and live instances, with results conditional on correct inverses, confinement, and its other stated premises. External emissions cross the boundary of what an inverse can restore, and untrusted code still needs a sandbox.

[DeepSeek Harness](#) [29] applies Cordis to an agent product: everything is a plugin, including model adapters, tools, skills, sessions, storage, sandboxes, workflows, the agent loop, and the interfaces. As of 21 August 2026 it calls itself a developer preview and warns that compatibility-breaking changes will occur. Its session is event-sourced: model-visible context is reconstructed from recorded entries, and a trajectory interface can inspect, resume, fork, and replay that history without repeating model or tool calls.

The honest contrast is scope and custody, not whether anyone keeps a record. Harness’s record covers the model-facing trajectory and session controls; its composition lives in layered configuration and the live Cordis assembly. Tenant identity, principal attribution, authority over attachments, placement across a fleet, and one execution model for strict and adaptive work need a wider runtime boundary. Cordis does have capability-style, proxy-mediated access; the narrower observation is that its formal center is component

lifecycle, not a principal and an execution record. And neither system's public materials make the tenant, principal, and placement guarantees Verlet is designed to make, which is a statement about what is claimed, not about what is impossible.

There is plenty to learn from them. Plugin boundaries have to be easy to use. The trajectory is a product surface, and fork, replay, and search make the record tangible in a way an audit API never will. The harness itself must be inspectable and changeable without reading the whole application. And an integration should target a tagged, stable surface rather than chase a pre-release contract.

Where the other products sit

Model providers supply the “maybe” computer; the runtime uses their capabilities and keeps durable identity outside any one of them. Sandboxes supply isolation, which is a placement choice, not an authority decision. Tool vendors supply operations, published immutably or surfaced through a search-and-call boundary with call-time validation. Registries distribute definitions and resolve names, while the runtime still pins the immutable identity before running. Governance and compliance products supply organizational judgment and control mapping, through the two seams in section 6. Observability and evaluation products build views over the record; a model-backed evaluation is itself a produced event with provenance. Orchestrators decide when and why agents run across a fleet, and should be clients of the runtime record rather than a second source of truth.

ASK YOUR AGENT

Describe the seams this runtime exposes to outside products: how an external policy service would be called on a consequential action and what it receives, and how events and receipts can be exported to an outside evidence or compliance system. For each seam, say whether it is implemented, partly implemented, or only designed, and point at the code or docs that show the current state.

10. Status and receipts

Verlet is experimental and says so. Three registers, kept separately so the architecture story does not imply production maturity.

Available in the current runtime (v0.4.0). Append-only event streams and receipts. Content-addressed operation publication. Journaled attach and detach with attachment-

carried configuration. Durable resume from the record. Agent manifest planning and publishing. Wasm operations and custom execution lanes. Virtual shell and filesystem surfaces. CLI, daemon, RPC, MCP, and ACP interfaces over shared contracts. Durable process and child-thread handles. Remote child placement through authenticated store-backed queues. Multi-instance host primitives with instance-owned authentication. (Checked against the v0.4.0 tag [28], 21 August 2026; the vocabulary is the formalism's [27].)

In progress. The pluggable policy router with held approval and resume. Stable evidence-export profiles for external governance systems. The preset and opening-sequence authoring experience. Managed-cloud operational hardening. Richer package and distribution surfaces. Stronger placement backends.

Direction. Self-serve local-to-managed promotion. Public and private package ecosystems. Fleet control and placement mobility. Governed self-extension end to end, from proposal through evaluated deployment. Conformance batteries for the runtime and evidence guarantees. Stable bridges to external harness formats.

The criteria

The claims above are checkable. The companion paper states thirteen yes/no criteria that decide whether any system actually works the way this primer describes, answerable from its records and configuration alone, and scores ten shipping systems plus Verlet against them. Nobody passes all thirteen, including Verlet; the paper says exactly which are met, which are partial, and which are only specified. The current self-score:

#	Criterion, in plain words	Verlet
C1	History is append-only and enough to rebuild state	partial
C2	Every event says whether it was witnessed or produced, with provenance	met
C3	Nothing deletable holds truth	partial
C4	Replay re-derives; it never re-runs the model or the world	met for resume, partial for full replay
C5	A fork is a shared prefix, nothing copied	met

#	Criterion, in plain words	Verlet
C6	The active system can be read as a wiring diagram	met
C7	Authority is attachment and mediation; not attached means absent	partial
C8	Every activation has a witnessed origin and a budgeted end	partial
C9	No side door: every continuation passed the checkpoint	met
C10	Strict or adaptive is a per-step dial on one record	specified
C11	Every tool row is a faithful surface of an immutable contract	met
C12	Every model call has a receipt for what it was shown	met
C13	The envelope exports, and placement changes nothing	partial

Six met, six partial, one specified. The runtime's test suite is organized around the same criteria, so "the record cannot have holes" points at a battery rather than a sentence.

The next proof is end to end

The next stage should be judged by one complete path rather than isolated primitives: author an agent locally, publish its operations, promote it to managed placement, hit an external policy service on a consequential action, record the decision and outcome, export the evidence, kill the runtime, and resume with the same identity and attachments. That single path tests ergonomics, runtime truth, governance integration, portability, and recovery together.

Between two failure modes. Self-modifying systems in the literature sit at two extremes. The Gödel machine (Schmidhuber, 2003 [21]) requires a proof that a change is an improvement before applying it: formally clean, never built. Prompt-evolution loops such as GEPA [22] and fine-tuning loops modify themselves with no durable ground truth, no receipts, and no reproducible evaluation: practical, and unaccountable, because they cannot hill-climb a landscape they cannot stand still on. The design here is the middle: changes are proposals on the record, accepted through a checkpoint, with the episodes that motivated them attached and a detach always available. No proof of improvement, but a complete account of what was tried and why. The status table above should be read with that framing; what is listed as partial is mostly the acceptance machinery for that middle path, not the record it rests on.

ASK YOUR AGENT

List the thirteen criteria this runtime says it measures itself against (look for the criteria in the docs or the test suite organization), and for each one point me at the test or battery that checks it. Then tell me which of the capabilities listed as “available” in the primer you can confirm from a current checkout and which you cannot.

11. Ten questions for any agent system

Use these without adopting Verlet’s vocabulary.

1. What is the unit a developer authors, and can the effective agent be inspected without reading application code?
2. Which facts are authoritative after a crash, and can the system restart without regenerating model output?
3. How is an ambiguous external effect (the payment call that may or may not have landed) resolved?
4. Can derived views be deleted and rebuilt from durable history?
5. Which named identity submitted the work, and does every effect trace back to it?
6. Where does a policy decision meet the execution path, and does the record connect the decision to the resulting action?
7. Can an agent widen its own authority without an external decision?

8. How does a composition change take effect for work already in progress, and can the system say which exact versions a turn used?
9. Can the same definition run locally and in managed placement, and what cannot move?
10. Which system owns desired state, which owns executed state, and can an evidence product point back to the source record?

Appendix: glossary

Friendly term	Runtime meaning
agent definition	versioned declaration that begins a thread with an opening set of attachments
tool	model-visible surface of an executable operation
operation	immutable executable contract with typed input, output, effects, and requirements
install or enable	attach a published operation to a scope under policy
disable or revoke	detach the operation or narrow its attachment
thread	durable identity and ordered history of one scope of work
turn	one submitted continuation cycle ending at quiescence or budget
context	deterministic assembly of selected facts and views for a model call, with a receipt
memory	product term for durable facts, produced summaries, retrieval views, and context sources
workflow	strict policy over operations and agents
agent	adaptive policy connected to a durable thread and its attachments; the propagator on that thread

Friendly term	Runtime meaning
coupling	any function acting on a thread, with declared reads, writes, and trigger
propagator	coupling that writes back into the stream it reads; the agent loop
projection	coupling that derives output elsewhere (summary, index); lossy, with provenance
controller	coupling that writes into a control stream; what products call a hook
chaotic	a coupling whose output cannot be recomputed from its inputs; model-backed by definition
subagent	delegated child thread with its own durable identity and outcome
principal	the human or service identity on whose authority work entered
receipt	durable explanation of what the runtime resolved or did
placement	selected execution backend and runtime instance
registry	catalog and distribution system for immutable definitions and aliases
evidence view	searchable or mapped representation derived from runtime facts

References

Works cited

1. Babai, L. (1985). Trading group theory for randomness. *Proceedings of the 17th ACM Symposium on Theory of Computing*, 421–429.
2. Shamir, A. (1992). $IP = PSPACE$. *Journal of the ACM*, 39(4), 869–877.

3. Goldwasser, S., Micali, S., and Rackoff, C. (1989). The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1), 186–208.
4. Necula, G. C. (1997). Proof-carrying code. *Proceedings of the 24th ACM Symposium on Principles of Programming Languages*, 106–119.
5. Futamura, Y. (1971). Partial evaluation of computation process: an approach to a compiler-compiler. *Systems, Computers, Controls*, 2(5), 45–50. Reprinted in *Higher-Order and Symbolic Computation*, 12(4), 1999.
6. Jones, N. D., Gomard, C. K., and Sestoft, P. (1993). *Partial Evaluation and Automatic Program Generation*. Prentice Hall.
7. Makholm, H. (2000). On Jones-optimal specialization for strongly typed languages. *Semantics, Applications, and Implementation of Program Generation (SAIG 2000)*, LNCS 1924, 129–148.
8. Ashby, W. R. (1956). *An Introduction to Cybernetics*. Chapman and Hall.
9. Conant, R. C. and Ashby, W. R. (1970). Every good regulator of a system must be a model of that system. *International Journal of Systems Science*, 1(2), 89–97.
10. Wiener, N. (1948). *Cybernetics: Or Control and Communication in the Animal and the Machine*. MIT Press.
11. von Foerster, H. (2003). *Understanding Understanding: Essays on Cybernetics and Cognition*. Springer.
12. Plotkin, G. and Power, J. (2003). Algebraic operations and generic effects. *Applied Categorical Structures*, 11(1), 69–94.
13. Plotkin, G. and Pretnar, M. (2009). Handlers of algebraic effects. *Programming Languages and Systems (ESOP 2009)*, LNCS 5502, 80–94.
14. Swierstra, W. (2008). Data types à la carte. *Journal of Functional Programming*, 18(4), 423–436.
15. Kiselyov, O. and Ishii, H. (2015). Freer monads, more extensible effects. *Proceedings of the 8th ACM SIGPLAN Symposium on Haskell*, 94–105.
16. Wadler, P. and Blott, S. (1989). How to make ad-hoc polymorphism less ad hoc. *Proceedings of the 16th ACM Symposium on Principles of Programming Languages*, 60–76.
17. Verlet, L. (1967). Computer “experiments” on classical fluids. I. Thermodynamical properties of Lennard-Jones molecules. *Physical Review*, 159(1), 98–103.
18. Swope, W. C., Andersen, H. C., Berens, P. H., and Wilson, K. R. (1982). A computer simulation method for the calculation of equilibrium constants for the formation of physical clusters of molecules: application to small water clusters. *Journal of Chemical Physics*, 76(1), 637–649.

19. Laio, A. and Parrinello, M. (2002). Escaping free-energy minima. *Proceedings of the National Academy of Sciences*, 99(20), 12562–12566.
20. Frenkel, D. and Smit, B. (2002). *Understanding Molecular Simulation: From Algorithms to Applications*, 2nd ed. Academic Press.
21. Schmidhuber, J. (2009). Gödel machines: fully self-referential optimal universal self-improvers. In Goertzel, B. and Pennachin, C. (eds.), *Artificial General Intelligence*, Springer, 199–226. First circulated 2003 as arXiv:cs/0309048.
22. Agrawal, L. A., et al. (2025). GEPA: Reflective prompt evolution can outperform reinforcement learning. arXiv:2507.19457.
23. Fowler, M. (2005). Event Sourcing. <https://martinfowler.com/eaDev/EventSourcing.html>
24. Lamport, L. (1978). Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7), 558–565.
25. Chacon, S. and Straub, B. (2014). *Pro Git*, 2nd ed. Apress. <https://git-scm.com/book>
26. Radul, A. and Sussman, G. J. (2009). The art of the propagator. MIT CSAIL Technical Report MIT-CSAIL-TR-2009-002.

Software and documents referred to in the text

27. Verlet Formalism: laws, lexicon, and grounding notes.
<https://github.com/emotionscientific/verlet-formalism>
28. Verlet Kernel, v0.4.0. <https://github.com/emotionscientific/verlet-kernel>
29. DeepSeek Harness. <https://github.com/deepseek-ai/deepseek-harness>
30. Cordis. <https://github.com/cordiverse/cordis>
31. Cordiverse, *A Programming Paradigm for Spatiotemporal Composability*.
<https://github.com/cordiverse/paper>

Status note, 21 August 2026: DeepSeek Harness is a developer preview. Cordis is under active development. Verlet is experimental. Every runtime claim in this document was checked against the Verlet Kernel v0.4.0 tag on that date.